

SKILL SESSION 15&16

Session 15 :

TASK 1 : To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;
    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Combined Redirection Shell\\n");
    printf("Examples:\\n");
    printf("  cat < input.txt > output.txt\\n");
    printf("  ls > output.txt 2> error.txt\\n");

    while (1)
    {
        printf("combined-redirect$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char command[MAX_INPUT];
        strcpy(command, input);

        char input_file[MAX_INPUT] = "";
        char output_file[MAX_INPUT] = "";
        char error_file[MAX_INPUT] = "";
```

```

/* Find input redirection < */
char *in_redirect = strchr(command, '<');

if (in_redirect != NULL)
{
    char *start = in_redirect + 1;

    while (*start == ' ' || *start == '\t')
        start++;

    char *end = start;

    while (*end != '\0' &&
           *end != '>' &&
           !(end[0] == '2' && end[1] == '>'))
    {
        end++;
    }

    int length = end - start;

    if (length <= 0)
    {
        printf("Error: No input file specified.\n");
        continue;
    }

    strncpy(input_file, start, length);
    input_file[length] = '\0';

    char *space = strpbrk(input_file, " \t");
    if (space != NULL)
        *space = '\0';

    /* Remove < input.txt from command */
    memmove(in_redirect, end, strlen(end) + 1);
}

```

```

/* Find error redirection 2> */
char *err_redirect = strstr(command, "2>");

if (err_redirect != NULL)
{
    char *start = err_redirect + 2;

    while (*start == ' ' || *start == '\t')
        start++;

    char *end = start;

    while (*end != '\0' &&
           *end != ' ' &&
           *end != '\t')
    {
        end++;
    }

    int length = end - start;

    if (length <= 0)
    {
        printf("Error: No error file specified.\n");
        continue;
    }

    strncpy(error_file, start, length);
    error_file[length] = '\0';

    /* Remove 2> error.txt from command */
    *err_redirect = '\0';
}

/* Find output redirection > */
char *out_redirect = strchr(command, '>');

if (out_redirect != NULL)
{
    char *start = out_redirect + 1;

    while (*start == ' ' || *start == '\t')
        start++;
}

```

```

char *end = start;

while (*end != '\0' &&
      *end != ' ' &&
      *end != '\t')
{
    end++;
}

int length = end - start;

if (length <= 0)
{
    printf("Error: No output file specified.\n");
    continue;
}

strncpy(output_file, start, length);
output_file[length] = '\0';

/* Remove > output.txt from command */
*out_redirect = '\0';
}

/* Remove trailing spaces */
int len = strlen(command);

while (len > 0 &&
      (command[len - 1] == ' ' ||
       command[len - 1] == '\t'))
{
    command[len - 1] = '\0';
    len--;
}

char *args[MAX_ARGS];
parse_command(command, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\n");
    continue;
}

```

```

    if (fd_out < 0)
    {
        perror("open output");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd_out, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        close(fd_out);
        exit(EXIT_FAILURE);
    }

    close(fd_out);
}

if (error_file[0] != '\0')
{
    fd_err = open(error_file,
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd_err < 0)
    {
        perror("open error");
        close(fd_err);
        exit(EXIT_FAILURE);
    }

    if (dup2(fd_err, STDERR_FILENO) < 0)
    {
        perror("dup2 error");
        close(fd_err);
        exit(EXIT_FAILURE);
    }

    close(fd_err);
}

execvp(args[0], args);

perror("execvp");
exit(EXIT_FAILURE);
}

```

```

        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
    {
        waitpid(pid, NULL, 0);
    }
}

printf("Combined redirection shell terminated.\n");

return 0;
}

```

```

[(base) saiabburi@Charishma-MacBook-Air ~ % mkdir Skill8
[(base) saiabburi@Charishma-MacBook-Air ~ % cd Skill8
[(base) saiabburi@Charishma-MacBook-Air Skill8 % touch session15_combined_redirect.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % nano session15_combined_redirect.c
[

```

```
[(base) saiabburi@Charishma-MacBook-Air Skill8 % cc -Wall -Wextra -g session15_combined_redirect.c -o session15_combined_redirect
[(base) saiabburi@Charishma-MacBook-Air Skill8 % ls
session15_combined_redirect      session15_combined_redirect.c      session15_combined_redirect.dSYM
[(base) saiabburi@Charishma-MacBook-Air Skill8 % echo "Hello from Skill Week 8" > input.txt
[(base) saiabburi@Charishma-MacBook-Air Skill8 % echo "Hello from Skill Week 8" > input.txt
[(base) saiabburi@Charishma-MacBook-Air Skill8 % cat input.txt
Hello from Skill Week 8
[(base) saiabburi@Charishma-MacBook-Air Skill8 % ./session15_combined_redirect
Combined Redirection Shell
Examples:
  cat < input.txt > output.txt
  ls > output.txt 2> error.txt
combined-redirect$ cat < input.txt > output.txt

Execution plan:
Command: cat
Input: input.txt
Output: output.txt

combined-redirect$ cat output.txt

Execution plan:
Command: cat

Hello from Skill Week 8
```

```
combined-redirect$ ./session15_combined_redirect
```

Execution plan:

Command: ./session15_combined_redirect

Combined Redirection Shell

Examples:

```
cat < input.txt > output.txt
```

```
ls > output.txt 2> error.txt
```

```
combined-redirect$ ls > files.txt 2> errors.txt
```

Execution plan:

Command: ls

Output: files.txt

Error: errors.txt

```
combined-redirect$ exit
```

Combined redirection shell terminated.

```
combined-redirect$ cat files.txt
```

Execution plan:

Command: cat

```
errors.txt
```

```
files.txt
```

```
input.txt
```

```
output.txt
```

```
session15_combined_redirect
```

```
session15_combined_redirect.c
```

```
session15_combined_redirect.dSYM
```

```
combined-redirect$ cat errors.txt
```

Execution plan:

Command: cat

```
combined-redirect$
```

TASK 2 : To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

UW PICO 5.09

File: session15_complex.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50
#define MAX_COMMANDS 10

void parse_args(char *command, char **args)
{
    int i = 0;
    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Complex Pipeline and Redirection Shell\\n");
    printf("Example:\\n");
    printf("  cat < input.txt | grep Session > result.txt\\n");

    while (1)
    {
        printf("complex-shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char commands[MAX_COMMANDS][MAX_INPUT];
        int command_count = 0;

        char *token = strtok(input, "|");

        while (token != NULL && command_count < MAX_COMMANDS)
        {
            while (*token == ' ' || *token == '\\t')
                token++;
            commands[command_count][0] = *token;
            command_count++;
            token = strtok(NULL, "|");
        }
    }
}
```

```

        strcpy(commands[command_count++], token);
    }

    token = strtok(NULL, "|");

printf("\nExecution Plan\n");
printf("-----\n");

for (int i = 0; i < command_count; i++)
{
    printf("Command %d: %s\n", i + 1, commands[i]);
}

printf("-----\n");

int pipes[MAX_COMMANDS - 1][2];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) < 0)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
}

pid_t pids[MAX_COMMANDS];

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)

```

```
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pids[i] == 0)
    {
        char command[MAX_INPUT];
        strcpy(command, commands[i]);

        char *input_file = NULL;
        char *output_file = NULL;

        /* Input redirection */
        char *in_redirect = strchr(command, '<');

        if (in_redirect != NULL)
        {
            *in_redirect = '\\0';

            char *file = strtok(in_redirect + 1, " \\t\\n");

            if (file != NULL)
                input_file = file;
        }

        /* Output redirection */
        char *out_redirect = strchr(command, '>');

        if (out_redirect != NULL)
        {
            *out_redirect = '\\0';

            char *file = strtok(out_redirect + 1, " \\t\\n");

            if (file != NULL)
                output_file = file;
        }

        char *args[MAX_ARGS];

        parse_args(command, args);

        if (args[0] == NULL)
        {
            fprintf(stderr, "Error: Empty command.\\n");
            exit(EXIT_FAILURE);
        }

        /* Connect input from previous pipe */
        if (i > 0)
        {
            if (dup2(pipes[i - 1][0], STDIN_FILENO) < 0)
            {
                perror("dup2 input pipe");
                exit(EXIT_FAILURE);
            }
        }
    }
```

```

/* Connect input from previous pipe */
if (i > 0)
{
    if (dup2(pipes[i - 1][0], STDIN_FILENO) < 0)
    {
        perror("dup2 input pipe");
        exit(EXIT_FAILURE);
    }
}

/* Connect output to next pipe */
if (i < command_count - 1)
{
    if (dup2(pipes[i][1], STDOUT_FILENO) < 0)
    {
        perror("dup2 output pipe");
        exit(EXIT_FAILURE);
    }
}

/* File input redirection takes priority */
if (input_file != NULL)
{
    int fd = open(input_file, O_RDONLY);

    if (fd < 0)
    {
        perror("open input");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);
}

/* File output redirection */
if (output_file != NULL)
{

```

```

                                O_WRONLY | O_CREAT | O_TRUNC,
                                0644);

    if (fd < 0)
    {
        perror("open output");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);
}

/* Close all pipe descriptors */
for (int j = 0; j < command_count - 1; j++)
{
    close(pipes[j][0]);
    close(pipes[j][1]);
}

execvp(args[0], args);

perror("execvp");
exit(EXIT_FAILURE);
}
}

/* Parent closes pipes */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all children */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll pipeline processes completed.\n\n");
}

printf("Complex shell terminated.\n");

return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air Skill8 % touch session15_complex.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % nano session15_complex.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % cc -Wall -Wextra -g session15_complex.c -o session15_complex
(base) saiabburi@Charishma-MacBook-Air Skill8 % printf "Session 15\nHello World\nSession 16\nDSA Practical\n" > input.txt
(base) saiabburi@Charishma-MacBook-Air Skill8 % cat input.txt
Session 15
Hello World
Session 16
DSA Practical
(base) saiabburi@Charishma-MacBook-Air Skill8 % cat input.txt
Session 15
Hello World
Session 16
DSA Practical
(base) saiabburi@Charishma-MacBook-Air Skill8 % ./session15_complex
Complex Pipeline and Redirection Shell
Example:
[ cat < input.txt | grep Session > result.txt

```

```

complex-shell$ cat < input.txt | grep Session > result.txt

```

Execution Plan

```

-----
Command 1: cat < input.txt
Command 2: grep Session > result.txt
-----

```

All pipeline processes completed.

```

complex-shell$ exit
Complex shell terminated.

```

All pipeline processes completed.

```

complex-shell$ cat result.txt

```

Execution Plan

```

-----
Command 1: cat result.txt
-----

```

```

Session 15
Session 16

```

All pipeline processes completed.

Session 16 :

TASK 1 : To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Background Job Shell\\n");
    printf("Example: sleep 10 &\\n");

    while (1)
    {
        /* Check for completed background processes */
        int status;
        pid_t finished_pid;

        while ((finished_pid = waitpid(-1, &status, WNOHANG)) > 0)
        {
            printf("\\n[Background job completed] PID: %d\\n",
                finished_pid);
        }

        printf("job-shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        int background = 0;

        int len = strlen(input);
```

```

        printf("[Foreground job completed] PID: %d\n", pid);
    }
}

/* Wait for remaining background jobs */
while (waitpid(-1, NULL, 0) > 0)
{
}

printf("Background job shell terminated.\n");

return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air Skill8 % touch session16_background.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % touch session16_background.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % nano session16_background.c
(base) saiabburi@Charishma-MacBook-Air Skill8 % cc -Wall -Wextra -g session16_background.c -o session16_background
(base) saiabburi@Charishma-MacBook-Air Skill8 % ./session16_background
Background Job Shell
Example: sleep 10 &
job-shell$ echo Hello
Hello
[Foreground job completed] PID: 6212
job-shell$ sleep 5 &
[Background job started] PID: 6214
job-shell$ echo Background test
Background test
[Foreground job completed] PID: 6215

[Background job completed] PID: 6214
job-shell$

```

TASK 2 : To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50
#define MAX_JOBS 20

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_INPUT];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "RUNNING");

    printf("[%d] PID %d started: %s\\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}
```

```

void update_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].state, "RUNNING") == 0)
        {
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result > 0)
            {
                strcpy(jobs[i].state, "DONE");
            }
        }
    }
}

void show_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No active jobs.\n");
        return;
    }

    printf("\nJob Table\n");
}

```

```

    printf("Job ID\tPID\tState\t\tCommand\n");
    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%d\t%d\t%s\t\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].state,
            jobs[i].command);
    }

    printf("-----\n");
}

void remove_completed_jobs()
{
    int i = 0;

    while (i < job_count)
    {
        if (strcmp(jobs[i].state, "DONE") == 0)
        {
            printf("Removing completed job [%d] PID %d\n",
                jobs[i].job_id,
                jobs[i].pid);

            for (int j = i; j < job_count - 1; j++)
            {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("Job Table Shell\n");
    printf("Commands:\n");
    printf("  command & -> background job\n");
    printf("  jobs       -> show job table\n");
    printf("  cleanup    -> remove completed jobs\n");
    printf("  exit       -> quit\n");

    while (1)
    {
        update_jobs();

        printf("job-table$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
    }
}

```

```
input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (strcmp(input, "jobs") == 0)
{
    show_jobs();
    continue;
}

if (strcmp(input, "cleanup") == 0)
{
    remove_completed_jobs();
    continue;
}

if (strlen(input) == 0)
    continue;

int background = 0;

int len = strlen(input);

while (len > 0 &&
        (input[len - 1] == ' ' ||
         input[len - 1] == '\t'))
{
    input[len - 1] = '\0';
    len--;
}
```

```

        waitpid(pid, NULL, 0);

        printf("Foreground job completed. PID: %d\n", pid);
    }
}

/* Wait for remaining jobs */
for (int i = 0; i < job_count; i++)
{
    if (strcmp(jobs[i].state, "RUNNING") == 0)
    {
        waitpid(jobs[i].pid, NULL, 0);
    }
}

printf("Job table shell terminated.\n");

return 0;
}

```

```

[(base) saiabburi@Charishma-MacBook-Air Skill8 % touch session16_job_table.c
[(base) saiabburi@Charishma-MacBook-Air Skill8 % nano session16_job_table.c
[(base) saiabburi@Charishma-MacBook-Air Skill8 % cc -Wall -Wextra -g session16_job_table.c -o session16_job_table
[(base) saiabburi@Charishma-MacBook-Air Skill8 % ./session16_job_table

```

Job Table Shell

Commands:

```

    command & -> background job
    jobs      -> show job table
    cleanup   -> remove completed jobs
    exit      -> quit

```

```

job-table$ sleep 10 &
[1] PID 6306 started: sleep 10
job-table$ sleep 10 &
[2] PID 6308 started: sleep 10
job-table$ jobs

```

Job Table

Job ID	PID	State	Command
1	6306	DONE	sleep 10
2	6308	RUNNING	sleep 10

```
job-table$ jobs
```

Job Table

Job ID	PID	State	Command
1	6306	DONE	sleep 10
2	6308	DONE	sleep 10

```

job-table$ cleanup
Removing completed job [1] PID 6306
Removing completed job [2] PID 6308
job-table$ jobs
No active jobs.
job-table$ exit
Job table shell terminated.

```

```
[(base) saiabburi@Charishma-MacBook-Air Skill8 % ls -lh
```

```
total 408
```

```
-rw-r--r-- 1 saiabburi staff    0B Sep 26 15:27 errors.txt
-rw-r--r-- 1 saiabburi staff 133B Sep 26 15:27 files.txt
-rw-r--r-- 1 saiabburi staff  48B Sep 26 15:44 input.txt
-rw-r--r-- 1 saiabburi staff  24B Sep 26 13:47 output.txt
-rw-r--r-- 1 saiabburi staff  22B Sep 26 15:44 result.txt
-rwxr-xr-x 1 saiabburi staff  34K Sep 26 13:46 session15_combined_redirect
-rw-r--r-- 1 saiabburi staff 6.6K Sep 26 13:46 session15_combined_redirect.c
drwxr-xr-x 3 saiabburi staff   96B Sep 26 13:46 session15_combined_redirect.dSYM
-rwxr-xr-x 1 saiabburi staff  34K Sep 26 15:44 session15_complex
-rw-r--r-- 1 saiabburi staff 5.9K Sep 26 15:43 session15_complex.c
drwxr-xr-x 3 saiabburi staff   96B Sep 26 15:44 session15_complex.dSYM
-rwxr-xr-x 1 saiabburi staff  34K Sep 26 17:38 session16_background
-rw-r--r-- 1 saiabburi staff 2.6K Sep 26 17:38 session16_background.c
drwxr-xr-x 3 saiabburi staff   96B Sep 26 17:38 session16_background.dSYM
-rwxr-xr-x 1 saiabburi staff  51K Sep 26 17:47 session16_job_table
-rw-r--r-- 1 saiabburi staff 5.0K Sep 26 17:47 session16_job_table.c
drwxr-xr-x 3 saiabburi staff   96B Sep 26 17:47 session16_job_table.dSYM
```

```
[(base) saiabburi@Charishma-MacBook-Air Skill8 % ls
```

errors.txt	session15_combined_redirect.c	session16_background.c
files.txt	session15_combined_redirect.dSYM	session16_background.dSYM
input.txt	session15_complex	session16_job_table
output.txt	session15_complex.c	session16_job_table.c
result.txt	session15_complex.dSYM	session16_job_table.dSYM
session15_combined_redirect	session16_background	

```
(base) saiabburi@Charishma-MacBook-Air Skill8 %
```